



Team DevAgentix

Usecase - 2
**Enterprise Knowledge Assistant (RAG +
Agentic Workflow)**

Low Level Design

Version 1.0

Date : 15-May-2026

Table of Contents

1. System Architecture Overview	3
1.1 Data Ingestion Layer (Background Service)	3
1.2 Agentic Reasoning Layer (The "Brain" Loop)	3
1.3 Retrieval & Reranking Layer (Precision Foundation)	4
1.4 Output & Interface Layer	4
1.5 Performance and Evaluation	4
1.6 Data Flow Diagram	5
2 API Specification (FastAPI)	6
2.1 Endpoint Definitions	6
2.1.1 POST api/v1/query or POST api/v1/query/stream (The Core Agentic Loop)	6
2.1.2 POST api/v1/ingest (Knowledge Base Management)	6
2.1.3 POST api/v1/feedback (RLHF Integration)	6
2.1.4 GET api/v1/evaluate (Continuous Quality Monitoring)	7
3 Detailed Data Schema: IT Ticket Knowledge Base	8
3.1. Core Data Fields (Vectorized, for CSV)	8
3.2. Structured Metadata (Filtering & Citations)	8
3.3 PostgreSQL Schema/ER Model	9
4 Data Ingestion Module (Background Service)	11
4.1 Multi-Source Data Connectors	11
4.2 Cleaning & PII Redaction	11
4.3 Processing & Embedding Pipeline	11
4.4 Vector Storage & Metadata Schema	12
4.5 Tool Logic: ticket_lookup	12
4.6 Data Linking Strategy (The "Multi-Hop" Advantage)	12
4.7 Data Ingestion Sequence	13
5 Agentic Reasoning Layer (LangGraph)	14
5.1 Core Orchestration Logic	14
5.2 Multi-Tool Integration	14
5.3 Multi-Hop Reasoning & Data Linking	15
5.4 Execution Flow (The "Brain" Loop)	15
5.5 LangGraph State Schema Definition	15
5.6 State Transition & Logic Flow	15
5.7 Data Persistence (Checkpointing)	19
5.8 Inference Sequence	19
6 Precision & Safety	20
6.1 Retrieval & Reranking Strategy	20
6.1.1 Stage 1: Candidate Retrieval (Vector Search)	20
6.1.2 Stage 2: High-Precision Reranking	20

6.1.3 Mathematical Thresholding & Guardrails	21
6.1.4 Validation Metrics	21
6.2 Logic & Guardrails	21
6.2.1 Mathematical Confidence Guardrails	22
6.2.2 Security & Compliance: PII Redaction	22
6.2.3 Hallucination Prevention (Faithfulness Guardrail)	23
6.2.4 Human-in-the-Loop (HITL) Escalation	23
6.2.5 Operational Guardrail: Latency Management (future enhancement)	23
7 Deployment & Tech Stack Summary	24
7.1 The "Precision" Stack	24
7.2 Deployment Infrastructure	24
7.3 Tech Stack Matrix	25
8 Data Source	26
9 Future Enhancements	27
9.1 Scalability: Decoupling the Ingestion Pipeline	27
9.2 Security: Identity-Aware Retrieval (RBAC)	27
9.3 Accuracy: Dynamic Confidence Thresholding	27
9.4 Reliability: The "Self-Correction" Loop	27

1. System Architecture Overview

The **System Architecture Overview** for the Enterprise Knowledge Copilot describes a stateful, modular, and agentic RAG (Retrieval-Augmented Generation) framework. This architecture is specifically designed to move beyond traditional "Naive RAG" by implementing a **ReAct (Reason + Act)** reasoning pattern to solve complex, multi-hop queries that bridge organizational knowledge silos.

The system is organized into four auditable, high-performance layers that separate data ingestion from real-time agentic reasoning and output generation.

1.1 Data Ingestion Layer (Background Service)

This layer transforms fragmented corporate data into a structured, searchable knowledge base.

- **Source Connectors:** Ingests data from PDFs, Wiki pages, SOPs, and historical IT support tickets.
- **Parsing & Recursive Chunking:** Uses **LangChain** loaders and recursive character text splitting to break documents into manageable chunks while preserving contextual integrity.
- **Embedding & Privacy:** Chunks are converted into vectors using the [all-MiniLM-L6-v2](#) or [bge-base-en-v1.5](#) models. Pre-ingestion cleaning includes **PII Redaction** to ensure compliance with global privacy standards like **GDPR** and India's **DPDP Act**.
- **Vector Database:** High-speed storage is handled by **Qdrant**, which stores embeddings alongside rich metadata for source traceability and citation.

1.2 Agentic Reasoning Layer (The "Brain" Loop)

Built on **LangGraph**, this layer manages the stateful orchestration of complex user sessions.

- **ReAct Agentic Pattern:** The system utilizes an orchestrator (primarily **Gemini**, with support for **Llama 3**) that autonomously determines the best tool to use based on the user's natural language query.

- **Multi-Tool Integration:** The agent can call upon three specialized tools:
 - **Tool 1: Document Search:** Performs semantic lookup within technical SOPs and PDF manuals.
 - **Tool 2: Ticket System Lookup:** Queries historical IT support tickets to find past resolutions.
 - **Tool 3: Summarizer:** Aggregates and distills information when the answer is spread across multiple sources.

1.3 Retrieval & Reranking Layer (Precision Foundation)

To meet high enterprise standards for accuracy, the system employs a two-stage retrieval process:

- **Candidate Retrieval:** Performs a Top-K similarity search in Qdrant to retrieve the initial **Top 20** chunks.
- **BGE-Reranking:** Integrates a **BGE-Reranker** to locally refine and prioritize the Top 3–5 most relevant chunks before passing them to the LLM. This ensures the system maintains a **Mean Reciprocal Rank (MRR) > 0.80**.

1.4 Output & Interface Layer

The final stage focuses on transparency, trust, and usability.

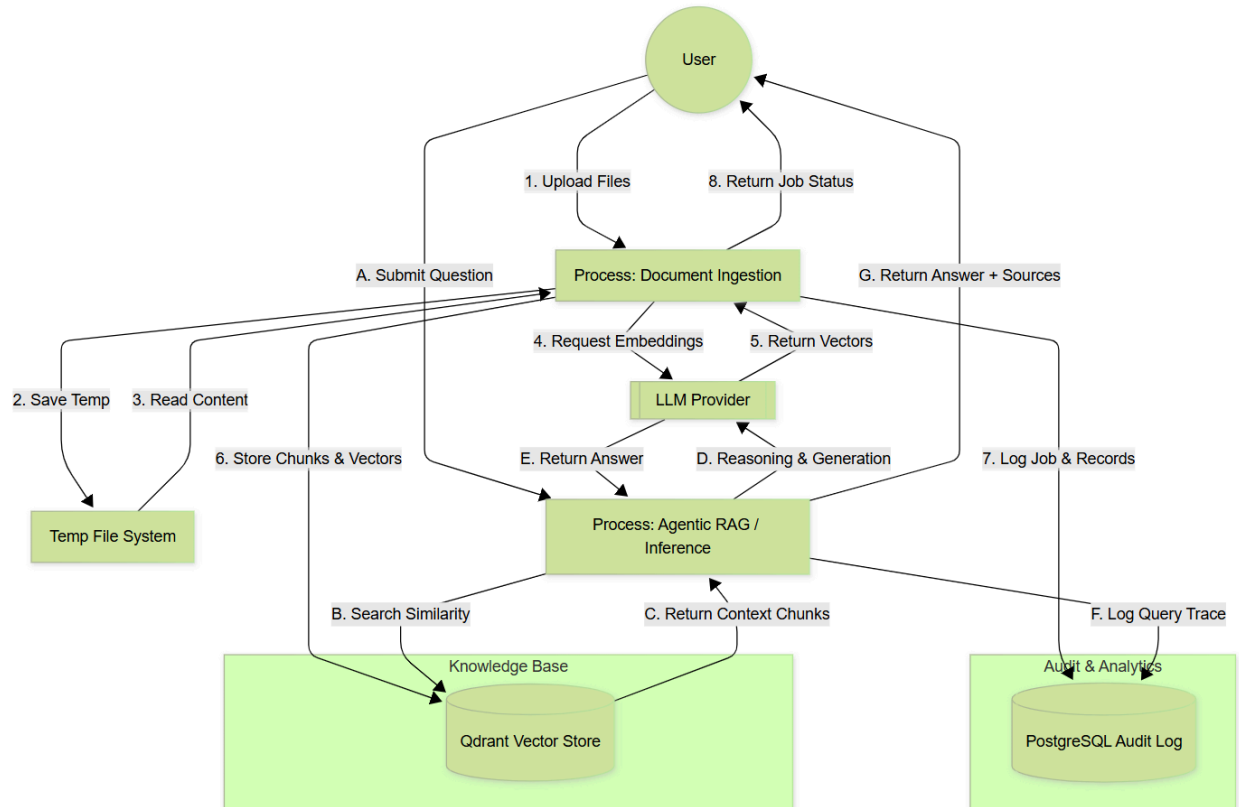
- **Grounded Generation:** The LLM generates a response based *only* on the verified chunks to prevent hallucinations.
- **Citation Injection:** Every claim in the response includes **inline source citations** (e.g., [\[Source: SOP_01\]](#)) derived directly from the chunk metadata.
- **Confidence Guardrails:** The system monitors the reranker's confidence score. If the score is **< 0.65**, it automatically returns "Answer not found" or escalates the query, ensuring 100% source verification for compliance.

1.5 Performance and Evaluation

The entire architecture is containerized using **Docker** and **Docker Compose** to ensure consistent deployment. Continuous monitoring is performed using **RAGAS** benchmarks to ensure **Answer Faithfulness > 0.85** and end-to-end latency **under 15**

seconds (which can further improve).

1.6 Data Flow Diagram



2 API Specification (FastAPI)

The backend is built using **FastAPI** due to its asynchronous capabilities, which are essential for handling long-running LLM and retrieval tasks without blocking the main thread.

2.1 Endpoint Definitions

2.1.1 **POST api/v1/query** or **POST api/v1/query/stream** (The Core Agentic Loop)

This is the primary endpoint for the Copilot. It initiates the LangGraph state machine.

- **Internal Logic:**
 - Validates the session via the LangGraph Checkpointer.
 - Triggers the **Reasoning Node** to determine if `doc_search` or `ticket_lookup` is needed.
 - Executes the **Reranking Node** to filter retrieved chunks.
 - Runs the **Grounded Generation Node** with citation injection.

2.1.2 **POST api/v1/ingest** (Knowledge Base Management)

Handles the asynchronous processing of new documents into the Qdrant vector store.

- **Request:** Multipart Form-Data (PDF/CSV/Text).
- **Logic:** Triggers the **Recursive Character Text Splitter** and embedding generation.
- **Response:** Returns an `ingestion_id` and the number of chunks successfully vectorized.

2.1.3 **POST api/v1/feedback** (RLHF Integration)

Captures user sentiment to refine the agent over time.

- **Request:** { "query_id": "...", "rating": 1, "comments": "Accurate solution." }
- **Logic:** Logs the data into a PostgreSQL table for future performance audits and fine-tuning.

2.1.4 GET api/v1/evaluate (Continuous Quality Monitoring)

This endpoint is designed for administrators and developers to audit the performance of the Copilot against the defined KPIs (Faithfulness > 0.85).

- **Purpose:** To perform a batch evaluation of the most recent interactions to detect "model drift" or quality degradation in specific knowledge domains.
- **Logic Flow:**
 - **Data Retrieval:** The system fetches the last 100 entries from the database (PostgreSQL). This includes the **question**, the **retrieved_context** (chunks from Qdrant), and the LLM's **final_response**.
 - **Dataset Preparation:** It formats this data into a "RAGAS-compatible" dataset (Columns: Question, Context, Answer).
 - **RAGAS Execution:** The endpoint triggers the RAGAS evaluation library to calculate:
 - **Faithfulness:** Is the answer derived *only* from the context?
 - **Answer Relevance:** Does the answer actually address the user's problem?
 - **Context Precision:** Were the top-ranked chunks actually the useful ones?
 - **Reporting:** It calculates the mean score for the batch.

3 Detailed Data Schema: IT Ticket Knowledge Base

This schema is designed to convert raw support logs into a "Resolution Library." Each record in the vector database (**Qdrant**) follows this structured format.

3.1. Core Data Fields (Vectorized, for CSV)

The following fields are concatenated or processed to create the embedding vector. This ensures that when a user asks about a "VPN error," the system matches the *semantic meaning* of the problem and the solution. For CSV file, it uses below fields and for unstructured contents like pdf, text files or markdown files, they are directly converted to embeddings and stored to the vector database.

Field Name	Data Type	Description
subject	String	A concise summary of the issue (e.g., "Intermittent VPN Disconnects").
description	Text	The full narrative of the user's problem, including error codes or symptoms.
resolution	Text	The verified fix applied by the engineer. This is the "Gold" data the agent seeks.

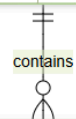
3.2. Structured Metadata (Filtering & Citations)

Metadata is stored as a payload in Qdrant to allow the Agent to apply filters (e.g., "Search only Networking tickets") and to provide accurate citations.

Metadata Key	Data Type	Purpose
<code>ticket_id</code>	String	Reference ID for the original ITSM record (e.g., INC-2023-882).
<code>category</code>	Enum	The domain of the issue (Networking, Hardware, IAM, Software).
<code>severity</code>	Integer	Historical priority (1-4) to help weight the importance of the fix.
<code>is_verified</code>	Boolean	Flag indicating if a Senior Engineer has audited this resolution.
<code>last_updated</code>	ISO Date	Ensures the agent prioritizes newer resolutions over obsolete ones.
<code>tags</code>	List[Str]	Keywords like <code>[Cisco, VPN, AnyConnect, Timeout]</code> .

3.3 PostgreSQL Schema/ER Model

INGEST_JOB			
uuid	id	PK	
string	status		pending running completed failed
int	total_files		
int	processed_files		
int	failed_files		
int	total_chunks		
text	error_message		
datetime	created_at		
datetime	completed_at		



DOCUMENT_RECORD			
uuid	id	PK	
uuid	job_id	FK	References INGEST_JOB.id
string	filename		
string	doc_type		sop ticket wiki
string	collection		Vector store collection name
int	chunk_count		
int	file_size_bytes		
string	sha256		
datetime	ingested_at		

QUERY_LOG			
uuid	id	PK	
text	question		
text	answer		
float	top_score		Confidence/Rerank score
boolean	escalated		
string	tool_used		
int	latency_ms		
datetime	created_at		
text[]	retrived_chunks		

4 Data Ingestion Module (Background Service)

The **Data Ingestion Module** functions as a robust background service designed to eliminate "tribal knowledge" silos by centralizing fragmented corporate data. This module transforms unstructured information from various sources into a high-precision, searchable vector format while ensuring strict enterprise compliance.

4.1 Multi-Source Data Connectors

The module utilizes specialized loaders to handle disparate enterprise data types:

- **Document Loaders:** LangChain-based parsers ingest technical SOPs and manuals stored in **PDF** format.
- **Wiki & Text Parsers:** Connectors for internal **Wiki pages** and HTML-based documentation.
- **Historical Ticket Importer:** Specifically handles **CSV/structured data** containing past IT support tickets and their associated resolution data.

4.2 Cleaning & PII Redaction

To maintain security and align with global privacy laws such as **India's DPDP Act** and **GDPR**, the ingestion service includes a mandatory cleaning phase:

- **Anonymization:** Personally Identifiable Information (PII) is redacted before data ever reaches the embedding model or vector store.
- **Data Validation:** Ensures that only readable, non-corrupted text with valid metadata is processed further.

4.3 Processing & Embedding Pipeline

Once cleaned, the data is prepared for semantic search through a two-step process:

- **Recursive Character Text Splitting:** This technique breaks down large documents into smaller, manageable chunks. By using recursive splitting, the system ensures that logical context is preserved across chunk boundaries,

which is critical for accurate retrieval.

- **Embedding Generation:** The module converts text chunks into numerical vectors using high-efficiency models like **all-MiniLM-L6-v2** or **bge-base-en-v1.5**. These models are chosen for their high performance-to-cost ratio and their ability to capture semantic meaning across technical IT domains.

4.4 Vector Storage & Metadata Schema

The final stage of ingestion involves storing the generated embeddings in **Qdrant**, a high-performance vector database.

- **Vector Search:** Qdrant facilitates rapid Top-K similarity searches, allowing the agent to retrieve relevant information in real-time.
- **Metadata Injection:** Every vector is stored with a detailed metadata payload. This schema is essential for the **Source Citation** requirement, as it tracks the original document name, page number, and source URL, ensuring 100% verification of generated answers during audits.

Component	Technology	Primary Role
Framework	LangChain	Orchestrating loaders and chunking logic.
Cleaning	Custom Middleware	Redacting PII for GDPR/DPDP compliance.
Model	all-MiniLM-L6-v2	Creating semantic vector representations.
Storage	Qdrant	Storing vectors with traceable metadata.

4.5 Tool Logic: **ticket_lookup**

Within the **Agentic Reasoning Layer**, the **ticket_lookup** tool is defined with specific Python logic to query this schema:

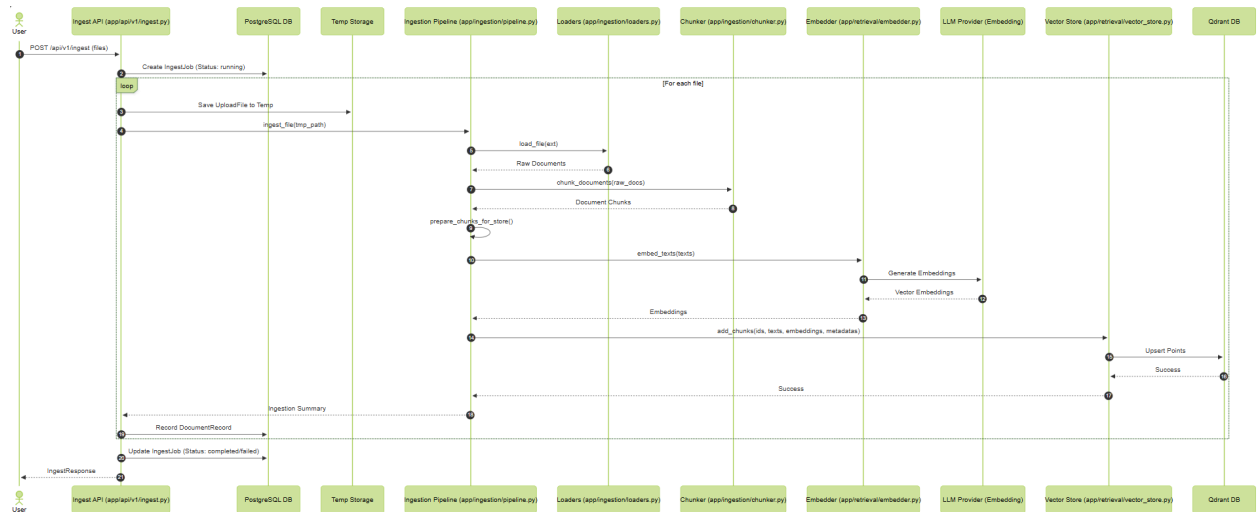
4.6 Data Linking Strategy (The "Multi-Hop" Advantage)

The LLD specifies a **Cross-Reference Logic**. When the agent retrieves a resolution

from a ticket, it attempts to link it back to an **SOP (Standard Operating Procedure)**:

- **Step A:** The agent finds a fix in a historical ticket ([INC-2023-882](#)).
- **Step B:** The agent checks the metadata [tags](#) of that ticket.
- **Step C:** The agent then calls the **Document Search Tool** to find the formal SOP that matches those tags.
- **Result:** The user receives a response that says: *"Based on past resolution [INC-2023-882], the fix is X. You can find the formal company policy in [SOP_Networking_v2]."*

4.7 Data Ingestion Sequence



5 Agentic Reasoning Layer (LangGraph)

The **Agentic Reasoning Layer** serves as the "Brain" of the Enterprise Knowledge Copilot. Unlike static RAG pipelines, this layer uses **LangGraph** to provide a stateful, iterative environment where the system can reason about a query before executing actions.

This layer moves beyond simple retrieval by implementing a **ReAct (Reason + Act)** pattern. The orchestrator does not just search; it evaluates if it has enough information to answer or if it needs to consult additional tools.

5.1 Core Orchestration Logic

- **Stateful Management:** LangGraph maintains the state of the conversation, allowing for complex, multi-turn dialogues. It tracks the history of the agent to ensure consistency across the session.
- **Model Agnosticism:** While currently utilizing **Gemini** for high-reasoning tool calls, the architecture is built to be model-agnostic, supporting open-source alternatives like **Llama 3** or **Mistral**.
- **Traceability:** Every decision—from choosing a tool to interpreting its output—is logged within the LangGraph framework, providing a full audit trail for enterprise compliance.

5.2 Multi-Tool Integration

The agent has access to a specialized toolkit, enabling it to route queries based on the nature of the request:

- **Tool 1: Document Search:** Interfaces with the Qdrant vector database to retrieve technical context from PDFs, Wikis, and SOPs.
- **Tool 2: Ticket System Lookup:** Specifically designed to query historical IT support tickets, allowing the agent to find precedents for technical issues.
- **Tool 3: Summarizer:** Used when information is retrieved from multiple sources and needs to be synthesized into a concise, readable resolution.

5.3 Multi-Hop Reasoning & Data Linking

A critical advantage of this agentic layer is its ability to perform **multi-hop analysis**. This allows the system to solve complex queries that a single search cannot answer, such as:

- Identifying a technical procedure from a **current SOP**.
- Linking that procedure to a **historical resolution** found in a support ticket to provide a real-world example.

5.4 Execution Flow (The "Brain" Loop)

- **Input:** The agent receives a natural language query.
- **Reasoning:** The agent "reasons" about which tool is required (e.g., "Is this a general policy question or a specific technical error?").
- **Action:** It calls the selected tool (e.g., `ticket_lookup`).
- **Observation:** The agent evaluates the tool's output. If the information is incomplete, it returns to step 2 to call another tool.
- **Final Response:** Once sufficient data is gathered, the agent generates a grounded answer with **inline citations**.

5.5 LangGraph State Schema Definition

The state is defined using a `TypedDict` in Python, which acts as the shared memory for all nodes in the graph.

5.6 State Transition & Logic Flow

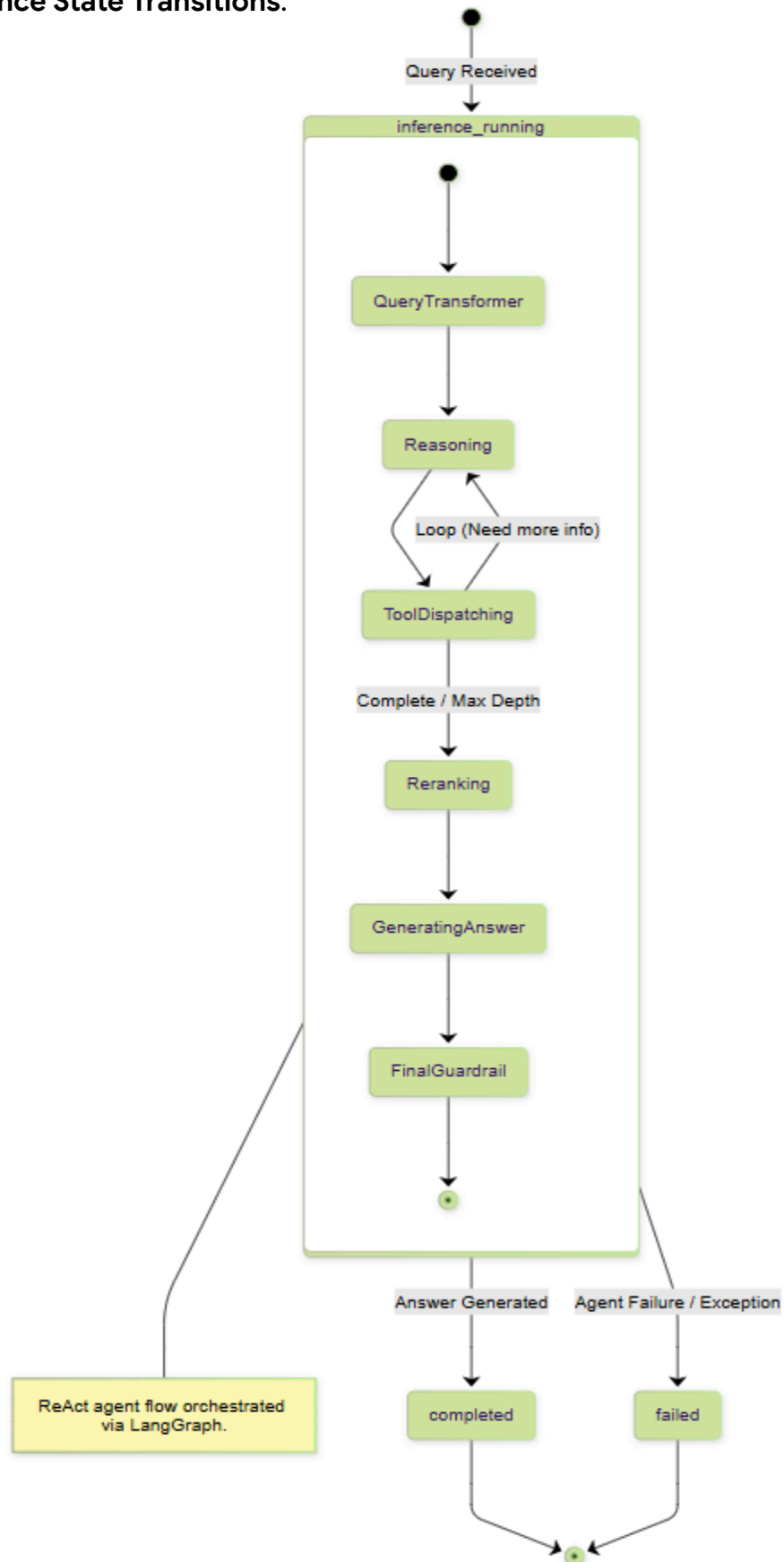
The state object is passed and modified through a series of specialized nodes.

- **Query Transformation Node:**
 - **Action:** Analyzes the `messages` history and sets the `search_query`.
 - **Logic:** If the user asks for a follow-up (e.g., "Tell me more about the first one"), this node re-writes the query to be self-contained.
- **Tool Selection Node (Conditional):**
 - **Action:** The LLM inspects the `search_query` and decides whether to update the state with `raw_documents` from the **SOP Search** tool or the

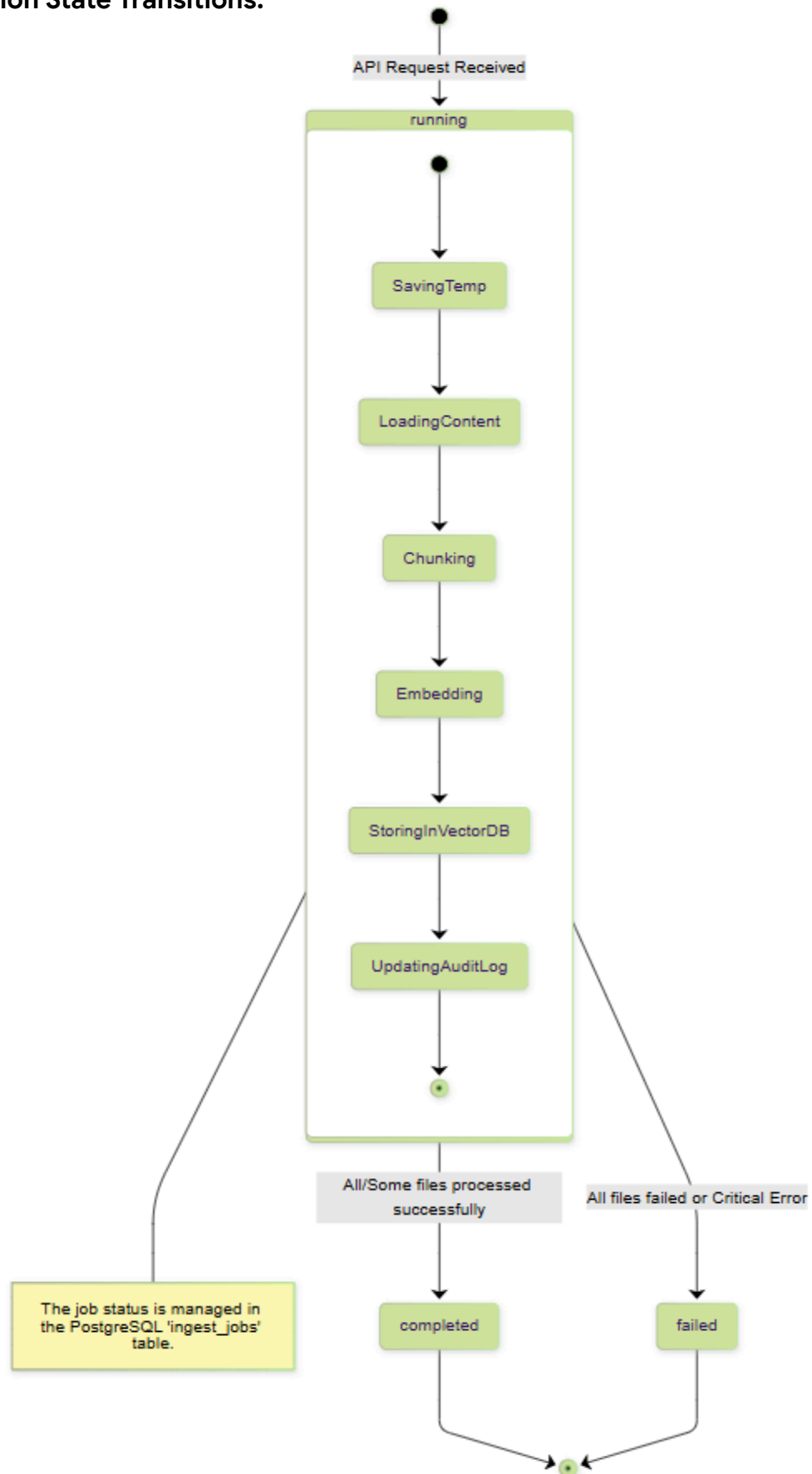
Ticket Lookup tool.

- **Reranking & Scoring Node:**
 - **Action:** Takes `raw_documents`, runs them through the **BGE-Reranker**, and updates `reranked_documents` and `max_confidence_score`.
- **Guardrail Router (The Critical Pivot):**
 - This is a conditional edge that reads `max_confidence_score` from the state:
 - **Score > 0.75:** Route to **Generation Node**.
 - **0.65 - 0.75:** Route to **Warning Node** (adds a "Proceed with caution" flag to the state).
 - **< 0.65:** Route to **Escalation Node** (sets `final_response` to "Answer not found" and triggers a support log).
- **Grounded Generation Node:**
 - **Action:** Consumes the `reranked_documents` to produce the `final_response`. It specifically looks at the metadata in the state to inject `[Source: ID]` markers.

- Inference State Transitions:



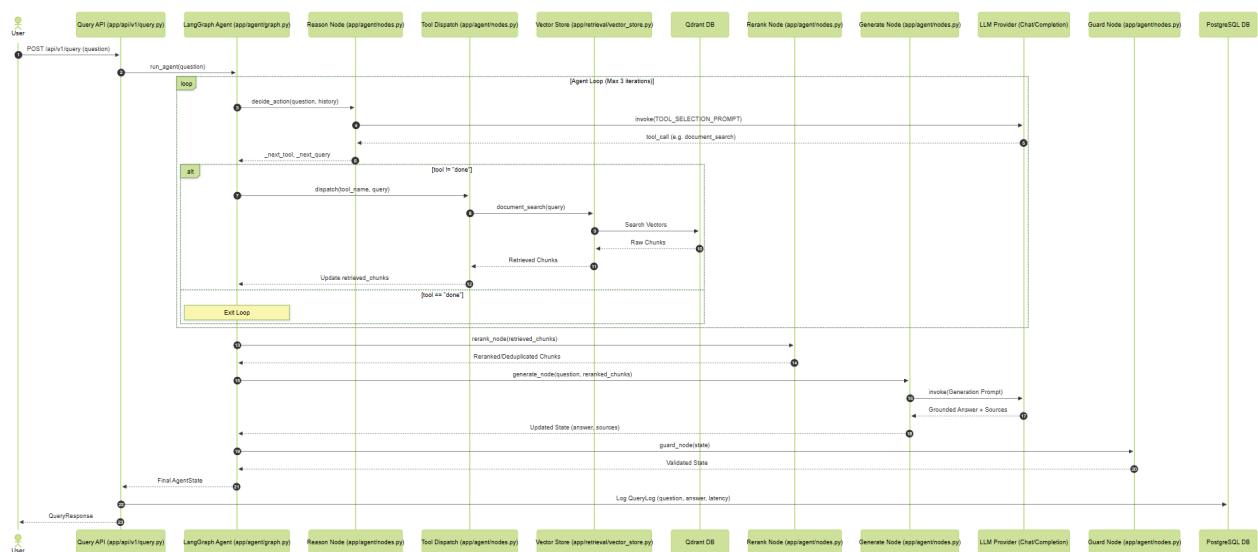
- Ingestion State Transitions:



5.7 Data Persistence (Checkpointing)

To handle multi-turn sessions, we implement a **PostgresSaver** checkpointer. This allows the LangGraph state to be saved and resumed using a `thread_id` and this `thread_id` would be `INGEST_JOB.id`, ensuring the Copilot remembers context even if the user refreshes their browser or returns after a short break.

5.8 Inference Sequence



6 Precision & Safety

6.1 Retrieval & Reranking Strategy

The **Retrieval & Reranking Strategy** is the core precision engine of the Enterprise Knowledge Copilot. By shifting from a "Naive RAG" approach to a two-stage retrieval process, the system ensures that the most semantically relevant information—rather than just the most similar text—is presented to the LLM for generation.

This strategy is designed to achieve high-precision results, targeting a **Mean Reciprocal Rank (MRR) > 0.80** and an **NDCG@5 > 0.75**.

6.1.1 Stage 1: Candidate Retrieval (Vector Search)

The initial retrieval phase focuses on recall to ensure no potential answer is missed.

- **Vector Database:** The system utilizes **Qdrant** for high-speed vector storage and retrieval.
- **Embedding Model:** Documents are embedded using **all-MiniLM-L6-v2** or **bge-base-en-v1.5**.
- **Initial Fetch:** The system performs a Top-K similarity search to retrieve the **Top 20** candidate chunks from the knowledge base.
- **Metadata Tracking:** Each chunk is retrieved alongside its metadata (source URL, page number) to support downstream citation injection.

6.1.2 Stage 2: High-Precision Reranking

Because vector similarity alone can be noisy, a second "reranking" stage is employed to refine the results.

- **BGE-Reranker (v1.5):** This specialized model evaluates the relationship between the user's query and each of the 20 candidate chunks.
- **Contextual Filtering:** The reranker re-orders the candidates based on true semantic relevance, selecting only the **Top 3–5** chunks to pass to the LLM.
- **Improvement Goal:** This two-stage process is specifically designed to improve **Top-1 retrieval** accuracy, ensuring the most correct answer appears first.

6.1.3 Mathematical Thresholding & Guardrails

To prevent hallucinations and maintain enterprise-grade trust, the system applies strict logic based on the reranker's confidence score.

Reranker Confidence Score (S)	System Action	Rationale
$S > 0.75$	Grounded Generation	High confidence; provide answer with inline citations.
$0.65 \leq S \leq 0.75$	Warning Label	Moderate confidence; show answer with a "low-confidence" warning.
$S < 0.65$	Escalation	Low confidence; return "Answer not found" and trigger Human-in-the-loop escalation.

6.1.4 Validation Metrics

The success of this strategy is measured using standardized benchmarks to ensure the system meets its performance targets:

- **Mean Reciprocal Rank (MRR):** Measures if the correct answer consistently appears at the top position (Target: **> 0.80**).
- **NDCG@5:** Measures the quality of the top 5 ranked results (Target: **> 0.75**).
- **Answer Faithfulness:** Verified via **RAGAS** to ensure the generated answer is strictly supported by the retrieved context (Target: **> 0.85**).

6.2 Logic & Guardrails

This section defines the multi-layered safety and reliability framework of the Enterprise Knowledge Copilot. In an agentic RAG system, **Logic & Guardrails** act as the "checks and balances" that prevent hallucinations, ensure data privacy, and maintain high-precision output.

The guardrail system is implemented as a sequence of interceptors within the LangGraph workflow, validating data at the Input, Reasoning, and Output stages.

6.2.1 Mathematical Confidence Guardrails

The system utilizes the BGE-Reranker's cross-encoder score as a quantitative measure of relevance. This score (S) is used to route the agent's logic:

- **High Confidence Logic ($S > 0.75$):**
 - **Action:** Proceed to standard grounded generation.
 - **Behavior:** The LLM is instructed to provide a definitive answer using the top-ranked chunks.
- **Moderate Confidence Logic ($0.65 \leq S \leq 0.75$):**
 - **Action:** Proceed with a "Cautionary Label."
 - **Behavior:** The system prepends the response with: *"I have found some information that may be relevant, but please verify with a senior engineer."* This manages user expectations for "fuzzy" matches.
- **Low Confidence Logic ($S < 0.65$):**
 - **Action:** Trigger "Knowledge Gap" protocol.
 - **Behavior:** The system bypasses LLM generation to avoid hallucinations and returns a standardized message: *"I am unable to find a definitive resolution in our current documentation. This query has been escalated for manual review."*

6.2.2 Security & Compliance: PII Redaction

To comply with **India's DPDP Act** and **GDPR**, the system implements a **Privacy Middleware** that operates before any data is processed by the LLM (Gemini/Llama 3).

- **Detection:** Uses a combination of Regex and a local Named Entity Recognition (NER) model (e.g., Presidio or Spacy) to identify names, email addresses, IP addresses, and phone numbers.
- **Transformation:** * **Input Level:** User queries containing sensitive data are masked (e.g., "How do I fix [NAME]'s laptop" ==> "How do I fix [USER]'s laptop").
 - **Storage Level:** During escalation, the raw query is redacted before being logged in the "Unresolved Queries" database to ensure no sensitive info is stored in plain text.

6.2.3 Hallucination Prevention (Faithfulness Guardrail)

Even with relevant chunks, LLMs can sometimes "hallucinate" or add external knowledge. We implement two specific checks:

- **Strict Context Adherence:** The system prompt forces the LLM into a "Closed Book" state: *"If the answer is not contained within the provided context tags , you MUST state that you do not know. Do not use your own internal training data to invent technical specs."*
- **RAGAS Evaluation Loop:** On the backend, periodic checks are run to calculate the **Faithfulness Score**. If the score for a specific category of questions (e.g., "Cloud Config") consistently drops below 0.85, the system flags that knowledge domain for re-ingestion or better chunking.

6.2.4 Human-in-the-Loop (HITL) Escalation

When the system fails to answer (Low Confidence), it triggers an automated escalation workflow:

- **Redacted Logging:** The query is stored in a ticket format.
- **Notification:** A webhook is sent to the IT Support Lead via Slack/Teams.
- **Feedback Loop:** Once the human lead provides the correct resolution, that resolution is converted into a **Mock Ticket** and fed back into the **Data Ingestion Module** (Section 2.1), allowing the Copilot to "learn" from the failure for future queries.

6.2.5 Operational Guardrail: Latency Management (future enhancement)

To ensure a **Response Latency < 15 seconds**, the logic includes a timeout guardrail:

- If the Reranker or LLM takes longer than 12 seconds to respond (considering network hopping / latency), the system triggers a fallback to the **Stage 1 (Naive RAG)** result with a disclaimer, ensuring the user is not left waiting indefinitely.

7 Deployment & Tech Stack Summary

The stack is selected to balance high precision (Reranking) with operational efficiency (Async Python).

7.1 The "Precision" Stack

- **Orchestration (LangGraph):** Unlike linear chains, LangGraph allows the system to "cycle back." If the ticket lookup yields no results, the agent can autonomously decide to try a broader document search.
- **Vector Database (Qdrant):** Chosen for its support for **Payload Filtering**. This allows us to filter search results by department or document type (e.g., "Only search SOPs for the Networking team") before the reranking step.
- **Reranker (BGE-Reranker-v1.5):** This is the "secret sauce" for high MRR. It is hosted as a local inference service (using [SentenceTransformers](#)) to avoid external API costs and latency.

7.2 Deployment Infrastructure

- **Containerization (Docker Compose):** The entire solution is defined in a [docker-compose.yml](#) file, ensuring that the development environment matches production exactly.
 - [app-service](#): FastAPI + LangGraph.
 - [qdrant-db](#): Vector storage.
 - [ui-service](#): ReactJS frontend.
- **Environment Management:** All configurations (API Keys for Gemini/Llama, Qdrant URLs, PII Redaction thresholds) are managed via a [.env](#) file for security and in case of cloud environment, we can use secret manager.
- **CI/CD Pipeline:** Integrated with **RAGAS** for automated testing. Every code push triggers a "mini-benchmark" to ensure that changes haven't degraded the **Answer Faithfulness** score below 0.85.

7.3 Tech Stack Matrix

Component	Technology Choice	Why?
Language	Python	Ecosystem support for AI/ML.
Framework	FastAPI	Async performance and auto-generated Swagger docs.
Agent Logic	LangGraph	Stateful, cyclic reasoning capabilities.
Embeddings	all-MiniLM-L6-v2	Lightweight and high-speed for local deployment.
Reranker	BAAI/bge-reranker-v1.5	State-of-the-art cross-encoder for precision.
Vector DB	Qdrant	Advanced filtering and high-dimensional search.
UI	ReactJS	Rapid prototyping for IT internal tools.
Evaluation Engine	RAGAS	Industry standard for reference-free evaluation of RAG pipelines.
Database (Relational)	PostgreSQL	Stores session history, feedback, and logs for the /v1/evaluate endpoint.
Container Orchestration	Docker Compose	Simplifies local hosting of FastAPI, Qdrant, and Postgres.
Model Hosting	Local/Cloud Hybrid	Gemini/Llama 3 for reasoning; local SentenceTransformers for embeddings and BGE-reranking.

8 Data Source

Category	Source Examples	Format
Unstructured Knowledge	SOPs, Manuals, Tech Specs, Design Docs	PDF, DOCX
Institutional Memory	Jira/ServiceNow Tickets, Zendesk Resolution Logs	CSV, JSON, API
Project Context	Confluence, internal Wikis, README.md	Markdown, HTML
Live Comms (Advanced)	Slack/Teams 'Help' Channels	JSON (Exports)

9 Future Enhancements

9.1 Scalability: Decoupling the Ingestion Pipeline

To ensure **scalability**, we can avoid process embeddings synchronously within the FastAPI worker. Instead implement a **Message Queue**. The API should return an `ingestion_id` immediately, and the background worker should handle the heavy lifting of documents parsing and embedding generation.

9.2 Security: Identity-Aware Retrieval (RBAC)

To make the system more **Compliant** and **Secure**, integrate **Role-Based Access Control (RBAC)** directly into the Qdrant retrieval step. Use the "Payload Filtering" capability of Qdrant to ensure that an HR user cannot retrieve IT Security SOPs, even if the semantic similarity is high.

9.3 Accuracy: Dynamic Confidence Thresholding

We can implement **Dynamic Thresholding** further. For example, like "General Wiki" queries, a 0.65 score is acceptable. For "Core Infrastructure" or "Security Resolution" tickets, the system should automatically raise the threshold to **0.80 or more than that** to ensure maximum accuracy in high-stakes domains.

9.4 Reliability: The "Self-Correction" Loop

We can further enhance the **LangGraph flow**. If the reranker returns a score below **Threshold**, instead of immediate escalation, the agent should attempt a **Query Rewriting** step (using a "Query Transformation Node") to see if a different semantic phrasing yields better chunks from the knowledge base.